

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ
по лабораторной работе № 6
по курсу «Разработка кроссплатформенных приложений»
Вариант 4

Выполнили
студенты группы 22ВОЭ1
Брюзгин А. С.
Тихонов Д. А.

Приняли
Юрова О. В.

Пенза 2025

Цель работы

Научиться создавать клиент-серверные приложения с использованием стандартных классов Java.

Задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру, обеспечивающую распределенное вычисление определенного интеграла на нескольких вычислительных узлах (клиентах) при этом каждый узел использует несколько нитей, как в предыдущей работе. Сервер не занимается вычислениями, а лишь реализует взаимодействие с пользователем и агрегацию результатов вычислений от клиентов.

Исходный код программы

```
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {  
  
    records.clear();  
    rectable.clear();  
  
    for (int j = 0; j != tModel.getRowCount(); j++) {  
        Double ul = (Double) jTable1.getValueAt(j, 0);  
        Double ll = (Double) jTable1.getValueAt(j, 1);  
        Double stp = (Double) jTable1.getValueAt(j, 2);  
  
        // Отправляем данные на сервер и получаем результат  
        Double rez = sendDataToServer(ul, ll, stp);  
  
        // Обновляем таблицу и записи  
        jTable1.setValueAt(rez, j, 3);  
    }  
}
```

```

    try {
        rectable.add(new RecTableBin(ul, ll, stp, rez));
        records.add(new RecIntegral(ul, ll, stp, rez));
    } catch (ErrRecIntegralVal ex) {
        Logger.getLogger(ContactEditorUI.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private Double sendDataToServer(Double ul, Double ll, Double stp) {
    try (Socket socket = new Socket("localhost", 12345);
        ObjectOutputStream output = new ObjectOutputStream(socket.getOutputStream());
        ObjectInputStream input = new ObjectInputStream(socket.getInputStream())) {

        // Отправляем данные на сервер
        output.writeDouble(ul);
        output.writeDouble(ll);
        output.writeDouble(stp);
        output.flush();

        // Получаем результат от сервера
        return input.readDouble();

    } catch (IOException e) {
        JOptionPane.showMessageDialog(null, "Ошибка подключения к серверу: " + e.getMessage(),
            "Ошибка", JOptionPane.ERROR_MESSAGE);
        return 0.0; // Возвращаем 0 в случае ошибки
    }
}

```

Выполнение программы

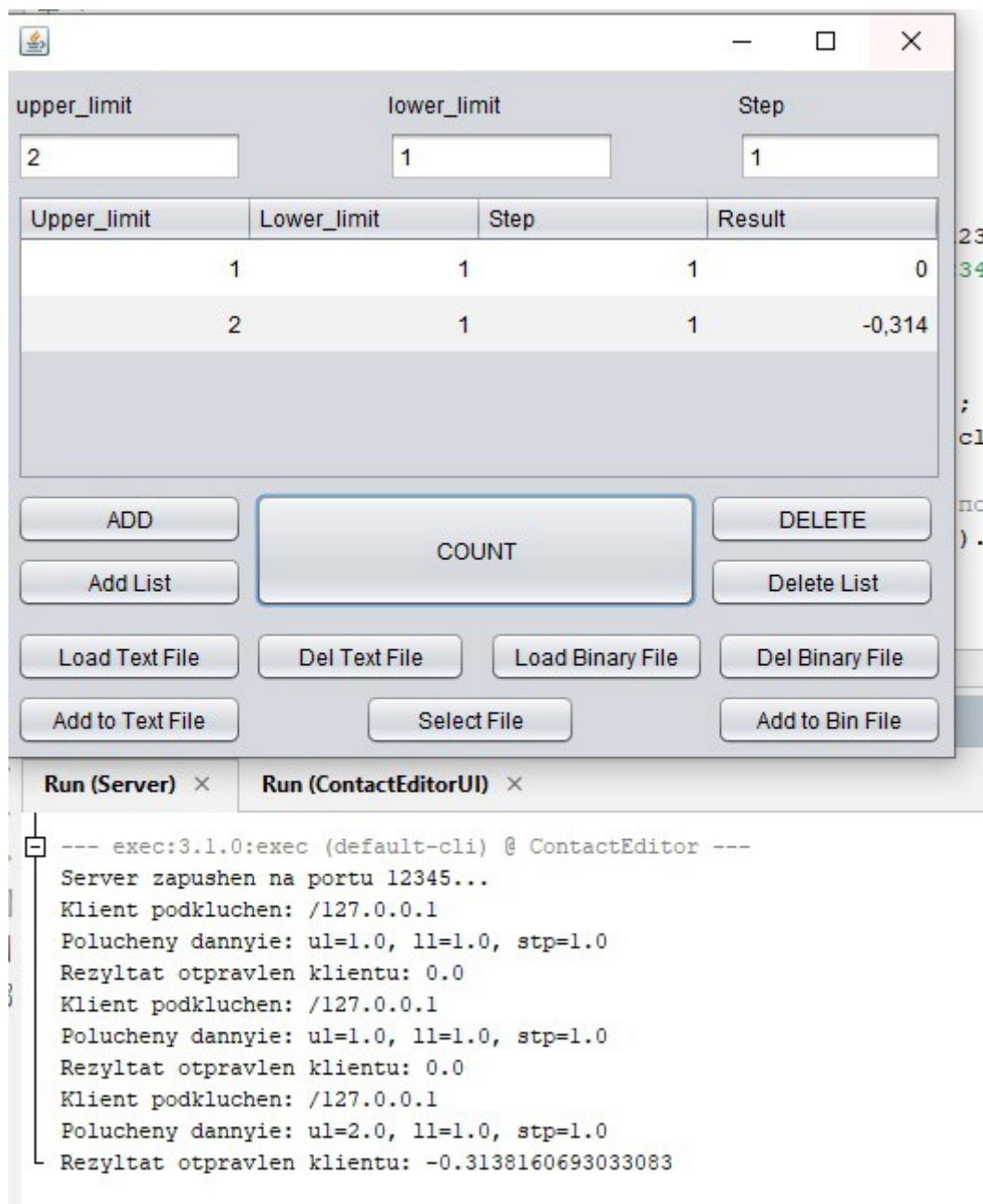


Рисунок 1 — Вычисление и работа сервера

Ход работы

```
/**
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */

package my.contacteditor;

import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.OutputStream;
import java.net.*;

/**
 *
 * @author art58
 */
public class Server {

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(12345)) {
            System.out.println("Сервер запущен на порту 12345...");

            while (true) {
                // Ожидаем подключения клиента
                Socket clientSocket = serverSocket.accept();
                System.out.println("Клиент подключен: " + clientSocket.getInetAddress());

                // Обрабатываем запрос клиента в отдельном потоке
                new Thread(() -> handleClient(clientSocket)).start();
            }
        } catch (IOException e) {
            System.err.println("Ошибка при запуске сервера: " + e.getMessage());
        }
    }
}
```

```

// // Метод для обработки запроса клиента
// private static void handleClient(Socket clientSocket) {
//     try (ObjectInputStream input = new ObjectInputStream(clientSocket.getInputStream());
//         ObjectOutputStream output = new ObjectOutputStream(clientSocket.getOutputStream())) {
//
//         // Читаем данные от клиента
//         double ul = input.readDouble();
//         double ll = input.readDouble();
//         double stp = input.readDouble();
//
//         System.out.println("Получены данные от клиента: ul=" + ul + ", ll=" + ll + ", stp=" + stp);
//
//         // Вычисляем интеграл
//         double result = calculateIntegral(ul, ll, stp);
//
//         // Отправляем результат клиенту
//         output.writeDouble(result);
//         output.flush();
//         System.out.println("Результат отправлен клиенту: " + result);
//
//     } catch (IOException e) {
//         System.err.println("Ошибка при работе с клиентом: " + e.getMessage());
//     } finally {
//         try {
//             clientSocket.close();
//         } catch (IOException e) {
//             System.err.println("Ошибка при закрытии сокета: " + e.getMessage());
//         }
//     }
// }
//
// // Метод для вычисления интеграла
// private static double calculateIntegral(double ul, double ll, double stp) {
//     double result = 0.0;
//     double n = (ul - ll) / stp;
//     double stp_ost = stp * (n - Math.floor(n));
//
//     if (Math.floor(n) == 0.0) {
//         for (double x = ll; x < ul; x += stp) {
//             result += (Math.tan(x) + Math.tan(x + stp)) * stp / 2;
//         }
//     } else {
//         double st = 0.0;

```

```

//      int k = 0;
//      while (k < Math.floor(n)) {
//          double osn1 = Math.tan(l1 + st);
//          double osn2 = Math.tan(l1 + st + stp);
//          double h = stp;
//          result += ((osn1 + osn2) * h) / 2;
//          st += stp;
//          k++;
//      }
//      double osn1 = Math.tan(l1 + st);
//      double osn2 = Math.tan(l1 + stp_ost);
//      double h = stp_ost;
//      result += ((osn1 + osn2) * h) / 2;
//  }
//
//      return result;
//  }
//}

```

Пояснение к тексту программы(основные вычисления)

Строки 1-44: Идет получение сервером данных клиента и настройка порта по которому работает сервер.

Строка 49: Вызов метода вычисления интеграла.

Строки 52-62: Идет отправка результатов обратно клиенту.

Строки 68-91: Вычисление интеграла.

Результат выполнения программы

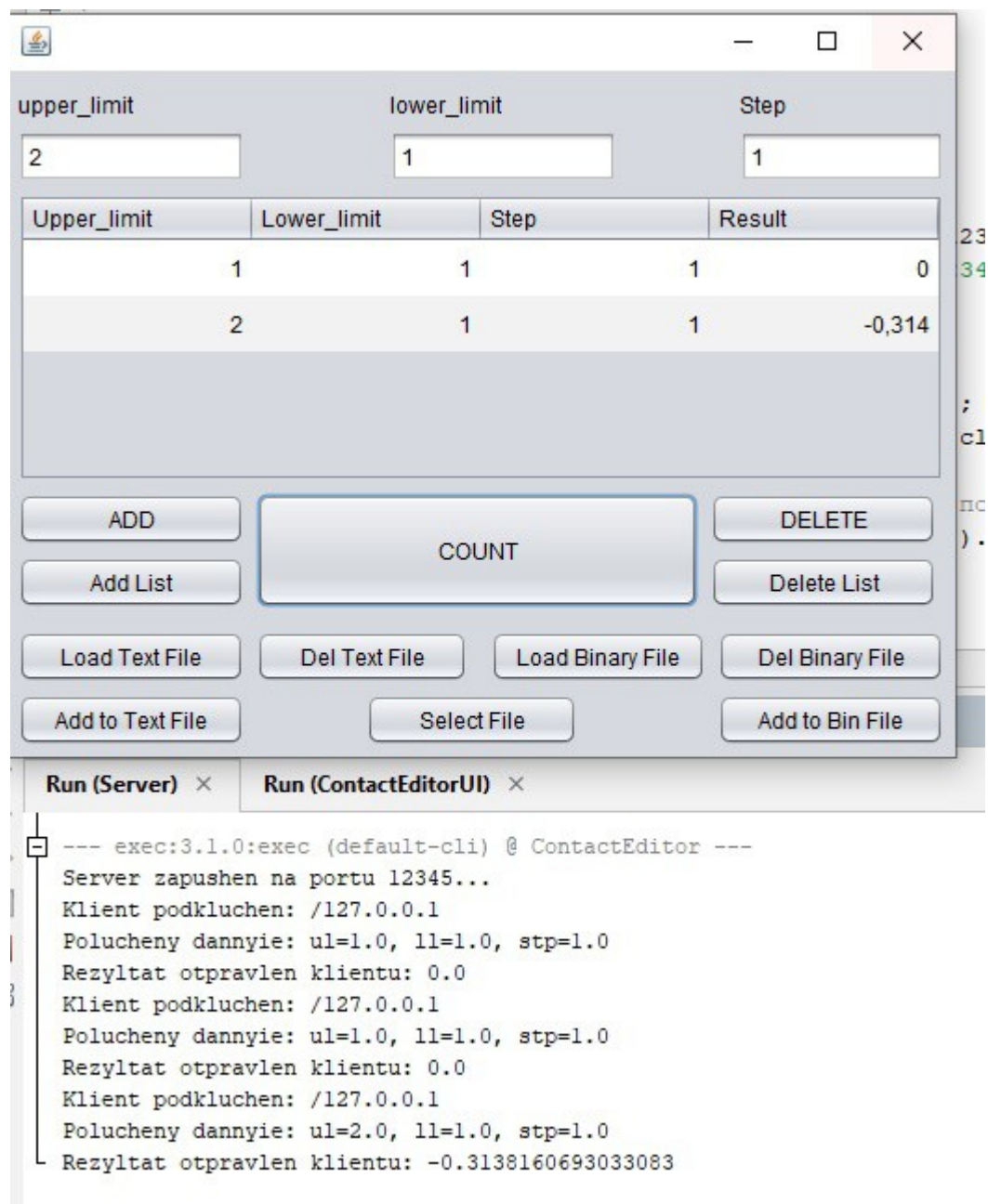


Рисунок 4 — Результат

Вывод

Изучена работа с серверами и создано клиент-серверное приложение.